

Project Mind Map Tool — V1 Brief / Initial PRD

1. Product Summary

This product is a **local, canvas-first project mapping tool** for understanding a codebase visually without being overwhelmed by a traditional IDE-style layout.

The core experience is a **whiteboard-style board** where the user explores a project through nodes and connections. The tool is meant to help the user quickly recover context when revisiting a project, understand what files are related, and create a semi-structured memory aid for how the project is organized.

This is **not** meant to be a full architecture truth engine, a full runtime tracer, or a heavy documentation platform in V1.

2. Core Goal

V1 should help the user answer questions like:

- What files or folders are directly related to this file?
- What direct connections does this node have?
- What note or explanation did I leave for this file?
- What packages/libraries are relevant here without flooding me with them?
- If this node has hidden related nodes elsewhere, how do I reveal them only when I want them?

The main design goal is:

Show only the necessary information by default, and reveal more only when the user asks for it.

3. V1 Product Identity

V1 is best understood as:

A visual repo memory tool

Not:

- a complete code intelligence system
- a full static-analysis engine
- a runtime execution tracer
- a giant IDE replacement

- a documentation wiki
-

4. V1 Scope

V1 is:

- local-first
- canvas-first
- one project at a time
- interactive
- semi-automatic
- manually augmentable
- lightweight in UI

V1 is not:

- multi-project linked map support
 - live auto-sync
 - always-on file watching sync
 - a VS Code extension first
 - a fully automatic architecture graph
 - a full end-to-end data-flow engine
 - a preview engine for web/native apps
 - a giant side-panel-heavy developer dashboard
-

5. Platform Direction

Recommended initial platform

Build V1 as a **standalone local app** rather than a VS Code-only extension.

Reasoning

The product is intended to: - work outside VS Code - still be useful when using another editor - survive profile switching - remain available even when the main editor is closed

A standalone-first build fits those goals better.

Recommended implementation direction

- Start as a **local web app** or a lightweight desktop-wrapped web app.
 - Keep the architecture compatible with later packaging or editor integration.
-

6. Main UX Direction

The product should behave like a **whiteboard / canvas tool**, closer in spirit to Miro/Figma-style spatial interaction than to a traditional IDE.

Important layout decision

There will be: - **no permanent left sidebar** - **no permanent right sidebar** - **no always-visible inspector panel**

Instead, the interface consists of: - a main canvas / whiteboard - an always-visible floating bottom dock - floating detail cards that appear only when needed

This keeps the app focused and avoids overwhelming the user.

7. Canvas / Whiteboard Behavior

Canvas model

The board is a **large bounded workspace**, not a tiny box and not an unconstrained infinite universe.

Reasoning

This keeps: - the user from getting lost in meaningless empty space - the viewport more manageable - rendering and navigation safer

Locked decisions

- The user can pan around the board.
- The board is bounded within a large world area.
- The board should not feel cramped.
- The board should not encourage endless empty wandering.

Performance philosophy

Performance should come primarily from: - rendering only what matters - not expanding too many nodes at once - avoiding constant relayout / physics simulation

Not merely from world bounds.

8. Main Interaction Model

Single-click on a node

Single-click reveals **only the direct relatives** of that node.

This means: - immediate neighbors only - one-hop reveal only - no recursive or full subtree expansion

Double-click on a node

Double-click opens a **floating detail card** for that node.

Double-click on empty canvas

If a new draft node is being created and is still empty, double-clicking empty canvas discards it.

Dragging

- dragging a node moves the node
- dragging the board pans the canvas
- floating detail cards can be dragged independently

Zooming

- zoom affects the board/canvas
 - floating detail cards stay screen-anchored and visible on screen
-

9. Detail Card Behavior

Detail cards replace the need for a permanent inspector.

Behavior

- opened by double-clicking a node
- appears near the node
- draggable
- remains in screen/UI space
- does not scale or move with the canvas like a node
- remains visible while the user pans/zooms the board

Purpose

Only show the minimum amount of detail needed for quick understanding.

Locked default contents for a file node

- file name
- shortened path, e.g. `../routes/connection.js`
- clicking the path reveals the full path
- copy path button
- short description (1–2 sentences only)

Additional small metadata allowed

- small node type badge, e.g. `File`
- small source badge, e.g. `AI` or `User`

Things intentionally excluded from the default detail card

- long metadata dumps
- imported-by lists
- large relationship lists
- package lists
- timestamps
- file size
- line counts
- heavy technical metadata

This is intentional to preserve clarity.

10. Node Types

A node represents a thing on the board.

Locked V1 node types

- `project`
- `folder`
- `file`
- `note`
- `package_group`
- `package`
- `hidden_connections_group`

Meaning

- **project**: the project root
- **folder**: a directory/folder
- **file**: a real file
- **note**: a manual note / explanation / concept node
- **package_group**: helper parent node like `Packages (8)`

- **package:** individual library/package node like `express` or `axios`
- **hidden_connections_group:** helper parent node like `Other connections (3)`

Helper nodes are intentional UI nodes used to reduce clutter.

11. Edge Types

An edge is a relationship line between nodes.

Locked V1 edge types

- `contains`
- `imports`
- `uses_package`
- `manual_link`
- `groups`

Meaning

- **contains:** structural relationship, e.g. project contains folder, folder contains file
 - **imports:** file imports another file
 - **uses_package:** file uses a library/package
 - **manual_link:** user-created non-structural relationship
 - **groups:** connection from a file/node to a grouping helper node like `Packages (x)` or `Other connections (x)`
-

12. Visual Edge Rules

Edge visuals should reflect **relationship meaning**, not just family-tree position.

Solid lines

Use solid lines for: - structural contains relationships - attached note relationships

Dashed / broken lines

Use dashed or broken lines for: - cross-file related links - sibling/cousin/non-structural links - manual cross-links - non-tree relationships

Important rule

Do **not** add duplicate shortcut structural edges if the visible structural chain already explains the path.

Example: If `project -> folder -> file` is already visible, do not also add a redundant direct structural `project -> file` edge unless it means something different.

Motion / routing rule

Cross-links may be routed in a way that reduces clutter, but nodes should not constantly reflow or nudge around aggressively. Prefer better edge routing over moving the graph too much.

13. Reveal Rules by Node Type

Core reveal rule

Single-click reveals only **direct relatives**.

Project node reveals

- direct child folders/files

Folder node reveals

- direct child folders/files

File node reveals

- direct notes
- grouped package node if relevant, e.g. `Packages (5)`
- grouped hidden-connections node if relevant, e.g. `Other connections (3)`
- direct descendants only where structurally relevant

Note node reveals

- directly linked nodes only

Package group node reveals

- individual package nodes only

Hidden connections group node reveals

- directly related file/note nodes that are not currently visible in their original structural path

Ancestor handling

Ancestors should **not** be auto-revealed on ordinary click.

Instead, ancestor context should be shown via a dedicated action such as: - `Reveal Path` -
`Show Parent Chain`

This keeps the graph cleaner.

14. Hidden Connections Design

If a node has direct relationships to other nodes that are not currently visible because those nodes live elsewhere in collapsed parts of the structure, V1 should not force those nodes into view immediately.

Instead, a helper node appears such as: - Other connections (x)

Behavior

- the helper node appears as the grouped placeholder
- clicking it reveals the connected nodes
- those nodes may initially appear detached from their full structural path
- only when the user explicitly chooses Reveal Path should the app show or unfold the ancestor chain for those nodes

This is a deliberate progressive-disclosure choice.

15. Package Reveal Design

Packages/libraries should not be dumped onto the canvas immediately for files that import many libraries.

Instead

The clicked file shows a grouped helper node such as: - Packages (2) - Packages (15)

Behavior

- click file node
- if package usage exists, reveal grouped package node
- click grouped package node
- reveal individual package nodes such as express, axios, etc.

This reduces clutter and prevents entry-point files from exploding visually.

16. Dock Design

The dock is the main control surface.

Dock philosophy

The dock should **launch actions**, not explain the project.

It should not become: - an explorer tree - an inspector panel - a metadata dump

Locked dock structure

The dock has two layers: - **Base dock** (always visible) - **Context dock** (appears when a node is selected)

Locked base dock items

- Search
- Scan
- Add
- Connect
- Pan
- Fit View
- Undo
- Redo
- Project / Settings

Locked context dock items

- Details
- Reveal Path
- Add Connection
- Collapse
- Center
- Hide

Locked dock arrangement

- **Left cluster:** navigation actions
- **Center cluster:** whiteboard actions
- **Right cluster:** system/project actions

This matches the intended mental model.

17. Search / Jump Behavior

Search is a primary navigation tool because there is no permanent explorer sidebar.

Locked behavior

- behaves like browser search suggestions
- as the user types, suggestions appear
- no automatic jump while typing
- pressing Enter or clicking a suggestion jumps to that node
- the chosen node becomes the center of the current view
- the node is highlighted

Additional intended behavior

If the node exists behind collapsed structural context, the app should reveal the minimum necessary context to make the jump useful.

18. Scan and Rescan Behavior

Locked scan philosophy

- V1 is **manual scan only**
- there is no live auto-sync in V1
- scan builds the **scanned** layer
- rescan updates the **scanned** layer and preserves the **user** layer

Locked user action

There is a visible `Scan` button in the dock (and optionally elsewhere in the viewport if desired later).

Locked scan UX direction

- clicking `Scan` triggers project update/scanning
- while scanning, the button should not launch parallel scans
- while scanning, the button should be disabled or otherwise guarded
- if no meaningful changes are found, the scan should exit quickly
- a visible scan animation is acceptable and desired, e.g. a wave animation, as long as it does not produce unnecessary system strain

Recommended initial scan algorithm

Use **iterative DFS** for filesystem traversal.

Reasoning

Iterative DFS is preferred over BFS for V1 because it: - fits natural tree traversal well - is easier to control with skip/opaque folder rules - keeps memory usage lower on very wide directory trees - is easier to stream incrementally

Scan structure

Use a **two-pass scan**.

Pass 1 — structure scan

DFS through the project tree and build: - project node - folder nodes - file nodes - opaque special folder nodes - `contains` edges

Pass 2 — relationship scan

For relevant scanned files, derive: - local file-to-file import relationships - package usage - grouped package helper nodes when needed - grouped hidden-connection helper nodes when needed

Locked auto-generated scanned content

The app may auto-create only objective structure such as: - project root - folders - files - structural `contains` relationships - local file-to-file import relationships - grouped package nodes - individual package nodes when revealed - grouped hidden-connections nodes when needed - opaque special folder nodes such as: - `node_modules` - `dist` - `build` - `.next` - `coverage` - `.git`

Opaque special-folder rule

Special heavy/noisy folders are represented as **single opaque nodes**.

Meaning

- they appear as single nodes
- they do not expand into their internal descendants in V1
- their detail card can explain what they are
- they remain part of the structure map without flooding the board

Locked manual-only content

The scan must **not** auto-create or overwrite user-owned meaning such as: - note nodes - manual links - aliases - descriptions - node positions - hidden states - collapsed states - manual semantic groupings - user-created conceptual flow nodes

Rescan merge philosophy

Rescan should behave as **incremental scanned-layer replacement with user-layer preservation**.

Locked rescan rules

- rescan rebuilds the new `scanned` snapshot
- the new `scanned` snapshot replaces the old `scanned` snapshot
- the `user` layer must be preserved
- if a scanned node disappears, that scanned node is removed
- if a scanned structural subtree disappears, its scanned descendants are also removed
- if a scanned file/folder is renamed, V1 treats this as **delete old + add new**
- V1 does not require smart rename detection

Orphan behavior

If user-created items still point to scanned nodes that disappeared: - the user-created items are preserved - those references are marked as `orphaned`, `detached`, or `unresolved` - the user may reconnect or delete them manually

Rescan diff strategy

Use **snapshot diff** / **manifest diff** rather than literal Git behavior.

Practical V1 diff basis

Compare old vs new scanned state using stable scanned identity such as: - node type - relative path from project root

Cheap change detection for V1

Use inexpensive metadata first where helpful, such as: - relative path - file type - modified time - file size

If nothing meaningful changed, skip deeper work.

19. Add Behavior

Locked behavior

Clicking **Add** should: 1. let the user choose the node type to add 2. create a new **draft node** at the center of the current viewport 3. auto-focus that node immediately 4. make it draggable

Draft node behavior

A newly created node begins as a **draft**, not a fully committed node.

Draft node rules

- it must be filled/saved or canceled
- it can be discarded by double-clicking empty canvas
- it should also support cancel via **Esc**
- while in creation mode, selection of other nodes is restricted until the draft is resolved

Save/cancel outcome

- if saved, the node becomes a real node
- if canceled/discarded while empty, it disappears

Creation follow-up mode

After a node is saved: - the node remains focused - the context dock becomes active - **Add Connection** should be emphasized/prominent - some context actions may be grayed out temporarily if they are not meaningful yet

Important UX stance

The system should strongly encourage the user to connect the new node, but not hard-force every possible next step in a frustrating way.

20. Connect Behavior

Locked V1 connect philosophy

Keep connect semantics very small in V1.

Locked manual connection types for V1

- Attached Note
- Related

Meaning

- **Attached Note:** a note attached closely to a node; visually treated like a tight annotation relationship
- **Related:** a general manual relationship between nodes that is not structural

Locked connect flow

1. User clicks **Connect**
2. User selects source node
3. User selects target node
4. App infers connection type by context where reasonable
5. Connection is created
6. Connect mode exits automatically after one connection

Intended default inference

- if either node is a note, default to **Attached Note**
- otherwise, default to **Related**

Visual rules

- **Attached Note** uses a solid line
 - **Related** uses a dashed/broken line
-

21. Reveal Path Action

A dedicated **Reveal Path** / **Show Parent Chain** action is part of the context dock.

Purpose

It provides structural context **only when the user asks for it**, especially for nodes revealed from hidden connection groups.

Reasoning

This avoids cluttering the board with full ancestor paths until needed.

22. Information Design Principles

These principles are now clearly established.

Principle 1 — Progressive disclosure

Do not show everything at once.

Principle 2 — Relationships first, structure on demand

The canvas should surface relationships in a compact way, while full path context can be explicitly revealed later.

Principle 3 — Minimal detail cards

Only show small, high-value metadata on demand.

Principle 4 — Group noisy relationship sets

Use helper nodes like: - Packages (x) - Other connections (x)

to avoid overwhelming the user.

Principle 5 — Manual augmentation matters

User notes and manual links are part of the product's value, not merely optional decorations.

23. Performance Principles

Exact profiling is deferred until implementation, but the following V1 principles are locked.

Performance direction

Keep the app light by: - avoiding fully expanded graphs by default - revealing only direct relatives - using grouped helper nodes to contain clutter - avoiding constant relay/layout/physics behavior - preferring manual scan over auto-sync - keeping detail cards small

Avoid

- constant auto-expansion
 - always-on global scanning
 - duplicate structural edges
 - too many visible cross-links at once
 - aggressive node motion
-

24. V1 User Outcome

A successful V1 should let the user: - open a project map - explore its structure visually without overload - reveal only immediate relationships on click - inspect only minimal detail on demand - add notes and manual links - see grouped package and hidden-connection indicators - reveal structural context only when needed - manually rescan the project when desired

25. Items Already Clarified / Locked

Locked platform/product direction

- standalone-first
- local-first
- one project at a time

Locked layout direction

- canvas-first whiteboard
- no permanent sidebars
- floating bottom dock
- floating screen-anchored detail cards

Locked interaction direction

- single-click reveals direct relatives only
- double-click opens details
- double-click empty canvas discards empty draft node

Locked helper node usage

- package grouping nodes

- hidden-connections grouping nodes

Locked dock items

Base dock: - Search - Scan - Add - Connect - Pan - Fit View - Undo - Redo - Project/Settings

Context dock: - Details - Reveal Path - Add Connection - Collapse - Center - Hide

Locked detail-card contents

- file name
- shortened path
- click-to-expand full path
- copy path
- 1-2 sentence description
- optional subtle type/source badge

Locked add/connect behavior direction

- Add creates draft node at viewport center
- Connect supports minimal manual relationship types in V1

26. Persistence, Storage, and Identity

Locked storage approach

V1 uses **local JSON-based persistence**.

Reasoning

JSON is sufficient for V1 because: - the tool is local-first - the data model is understandable and debuggable - user/project map data does not yet require relational querying complexity - it keeps implementation simple

Locked storage direction

- each tracked project should have its own project map file
- multiple JSON project-map files are acceptable
- no database server is required for V1
- no separate backend server is required for V1

Layering rule

The project map file should separate: - project/app metadata - scanned graph data - user-owned graph data

Stable scanned identity

For V1, scanned node identity should be based primarily on: - node type - relative path from project root

This is sufficient for incremental rescan behavior in V1.

27. Alias Support

Alias support is included in V1.

Purpose

An alias lets the user give a node a friendlier or more meaningful display label **without changing the real scanned name**.

Locked rule

- real scanned name remains source-of-truth
- alias is user-owned presentation metadata
- alias belongs in the `user` layer, not the `scanned` layer

Display rule

- if alias exists, show alias as the primary label on the board
 - the real scanned name/path remains available in the detail card
-

28. Future / Optional AI-Assisted Augmentation Direction

This is **not a core V1 feature**, but it is a valid future direction and should be acknowledged in the spec.

Intent

The user may optionally use an external AI tool manually to generate: - suggested descriptions - suggested note nodes - suggested manual relationship payloads - suggested higher-level semantic structure

The AI is not assumed to be built into the app in V1.

Important constraint

In V1, AI is **not** the primary source of truth for objective structure.

Objective structure should still come from the app's own scan logic for: - project root - folders - files - structural containment - local imports - package usage

Safe future role for AI

AI-generated payloads should be treated as: - optional user-imported suggestions - semantic augmentation
- boilerplate starting points

not as canonical authority.

Validation principle

If future AI-imported payloads affect structural graph elements, the app should validate them against the scanned repository before accepting them.

V1 stance

- No integrated AI graph-generation workflow is required in V1
- Manual prompting + manual copy/paste by the user is acceptable outside the product
- The API may exist later, but it is not necessary to rely on it for V1

29. JSON Shape and Schema Example

A clean V1 map file should use three root sections:

```
{
  "meta": {},
  "scanned": {},
  "user": {}
}
```

meta

Holds project-level metadata and scan config.

Suggested fields: - `schemaVersion` - `projectName` - `projectRoot` - `createdAt` - `updatedAt` - `lastScanAt` - `ignoreRules` - `specialOpaqueFolders`

scanned

Holds everything generated from repo scan.

Suggested fields: - `nodes` - `edges` - `warnings`

user

Holds everything owned by the user and preserved across rescans.

Suggested fields: - `nodeOverrides` - `manualNodes` - `manualEdges` - `uiState`

V1 JSON schema example

```
{
  "meta": {
    "schemaVersion": 1,
    "projectName": "my-app",
    "projectRoot": "/home/user/projects/my-app",
    "createdAt": "2026-04-03T12:00:00Z",
    "updatedAt": "2026-04-03T12:10:00Z",
    "lastScanAt": "2026-04-03T12:08:00Z",
    "ignoreRules": [
      "node_modules",
      "dist",
      "build",
      ".next",
      "coverage",
      ".git"
    ],
    "specialOpaqueFolders": [
      "node_modules",
      "dist",
      "build",
      ".next",
      "coverage",
      ".git"
    ]
  },
  "scanned": {
    "nodes": {
      "project:root": {
        "id": "project:root",
        "type": "project",
        "name": "my-app",
        "path": ".",
        "parentId": null
      },
      "folder:src": {
        "id": "folder:src",
        "type": "folder",
        "name": "src",
        "path": "src",
        "parentId": "project:root"
      },
      "file:src/server.js": {
        "id": "file:src/server.js",
```

```

    "type": "file",
    "name": "server.js",
    "path": "src/server.js",
    "parentId": "folder:src"
  },
  "special:node_modules": {
    "id": "special:node_modules",
    "type": "folder",
    "name": "node_modules",
    "path": "node_modules",
    "parentId": "project:root",
    "isOpaque": true,
    "description":
      "Library/package directory. Treated as one opaque node in V1."
  },
  "pkggrp:file:src/server.js": {
    "id": "pkggrp:file:src/server.js",
    "type": "package_group",
    "name": "Packages (2)",
    "ownerId": "file:src/server.js"
  },
  "package:express": {
    "id": "package:express",
    "type": "package",
    "name": "express"
  },
  "package:axios": {
    "id": "package:axios",
    "type": "package",
    "name": "axios"
  },
  "othergrp:file:src/server.js": {
    "id": "othergrp:file:src/server.js",
    "type": "hidden_connections_group",
    "name": "Other connections (1)",
    "ownerId": "file:src/server.js"
  }
},
"edges": {
  "edge:project:root->folder:src:contains": {
    "id": "edge:project:root->folder:src:contains",
    "type": "contains",
    "source": "project:root",
    "target": "folder:src"
  },
  "edge:folder:src->file:src/server.js:contains": {
    "id": "edge:folder:src->file:src/server.js:contains",
    "type": "contains",

```

```

    "source": "folder:src",
    "target": "file:src/server.js"
  },
  "edge:file:src/server.js->pkggrp:file:src/server.js:groups": {
    "id": "edge:file:src/server.js->pkggrp:file:src/server.js:groups",
    "type": "groups",
    "source": "file:src/server.js",
    "target": "pkggrp:file:src/server.js"
  },
  "edge:pkggrp:file:src/server.js->package:express:groups": {
    "id": "edge:pkggrp:file:src/server.js->package:express:groups",
    "type": "groups",
    "source": "pkggrp:file:src/server.js",
    "target": "package:express"
  },
  "edge:pkggrp:file:src/server.js->package:axios:groups": {
    "id": "edge:pkggrp:file:src/server.js->package:axios:groups",
    "type": "groups",
    "source": "pkggrp:file:src/server.js",
    "target": "package:axios"
  },
  "edge:file:src/server.js->othergrp:file:src/server.js:groups": {
    "id": "edge:file:src/server.js->othergrp:file:src/server.js:groups",
    "type": "groups",
    "source": "file:src/server.js",
    "target": "othergrp:file:src/server.js"
  }
},
"warnings": []
},
"user": {
  "nodeOverrides": {
    "file:src/server.js": {
      "alias": "Server Entry",
      "description": "Main backend entry point. Boots the app and wires key
server dependencies.",
      "descriptionSource": "user",
      "position": {
        "x": 420,
        "y": 210
      },
      "hidden": false,
      "collapsed": false
    }
  }
},
"manualNodes": {
  "note:create-document-flow": {
    "id": "note:create-document-flow",

```

```

    "type": "note",
    "name": "Create Document Flow",
    "description": "Tracks the main nodes involved when a document is
created.",
    "position": {
      "x": 650,
      "y": 260
    },
    "status": "active"
  }
},
"manualEdges": {
  "edge:note:create-document-flow->file:src/server.js:manual_link": {
    "id": "edge:note:create-document-flow->file:src/server.js:manual_link",
    "type": "manual_link",
    "source": "note:create-document-flow",
    "target": "file:src/server.js",
    "relation": "Attached Note",
    "orphaned": false
  }
},
"uiState": {
  "selectedNodeId": "file:src/server.js",
  "viewport": {
    "x": 0,
    "y": 0,
    "zoom": 1
  }
}
}
}

```

Notes on this schema

- scanned is replaceable on rescan
- user must be preserved across rescans
- alias belongs in user.nodeOverrides
- user-owned descriptions also belong in user.nodeOverrides
- manual notes and manual links live separately from scanned nodes/edges
- opaque folders are represented as scanned nodes with isOpaque: true

30. Implementation Guardrails for V1

Supported relationship-parsed file types

V1 officially supports relationship parsing for: - `.js` - `.jsx` - `.ts` - `.tsx`

Optional later additions may include: - `.mjs` - `.cjs`

Future candidate languages (not V1)

The following may be considered later, but are not part of V1 relationship parsing support: - C - Java - GDScript

Import detection scope

V1 supports only: - standard static ES module imports - CommonJS `require(...)`

V1 does **not** attempt to support: - dynamic imports - alias resolution - framework-specific import magic - custom bundler/module resolution tricks

Initial placement and reveal layout

- when a project is first opened, only the project root node is visible
- the root node appears at the center of the viewport
- no descendants are expanded by default
- descendants expand toward the right side of the canvas
- the visual fan-out direction should roughly follow northeast, east, and southeast
- ancestors remain conceptually to the left, descendants to the right

Helper-node placement

To improve readability, helper nodes should be placed consistently to the right side of their owner node with small positional offsets.

Suggested examples: - `Packages (x)` appears lower-right of the owner node - `Other connections (x)` appears upper-right of the owner node

Error and warning behavior

Fatal project-open error

If the project root cannot be read: - scanning stops - a blocking modal is shown - the message should clearly explain that the project could not be opened or scanned

Partial scan issues

If some folders, files, or import relationships cannot be processed: - scanning continues - issues are collected internally - the app does not expose raw terminal-style logs as the primary UI - a subtle warning indicator is shown - clicking the warning indicator opens a simple issue list

Limited-support language warning

If the project contains file types with limited V1 support: - the app should not broadly say the entire project is unsupported - instead it should state that some file types have limited support - structure can still be scanned, but relationship detection may be incomplete for those files

Project open behavior

- the app should always restore the last saved board state immediately when opening a known project
- V1 should not automatically reset the board back to root-only just because time has passed
- if the previous scan is stale, the app may show a gentle rescan prompt
- full rescan remains a manual action in V1
- V1 should not auto-run a full background rescan on open by default

31. Short One-Paragraph V1 Definition

V1 is a standalone, local-first, canvas-based project mapping tool for one codebase at a time. It presents a bounded whiteboard where users explore project structure and relationships through nodes, grouped helper nodes, and progressive disclosure. Single-click reveals only direct relatives, double-click opens a small floating detail card, and a floating bottom dock handles navigation, creation, connection, and project actions. The system is semi-automatic: it can scaffold structure and relationships from the project, while the user can add notes, manual links, and descriptions without being overwhelmed by a traditional IDE-like interface.